

LAPORAN ANALISIS PROGRAM SISTEM KLINIK SEDERHANA (SIMKLINIK)

(Studi Implementasi Java Swing dan MySQL)

Mata Kuliah Pemrograman Berbasis Objek (Java)

Dosen Pengampu: Yulhan Wahyudin, M.Kom



Disusun oleh:

Nama : Ari Rizal Zalaludin

NIM : 20240040042

Kelas : TI24M

**FAKULTAS TEKNIK KOMPUTER DAN DESAIN
PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS NUSA PUTRA
2026**

DAFTAR ISI

BAB I DESKRIPSI PROGRAM	1
1.1 Nama Aplikasi	1
1.2 Tujuan Aplikasi	1
1.3 Permasalahan yang Diselesaikan.....	1
BAB II ANALISIS KONSEP PEMROGRAMAN BERORIENTASI OBJEK (PBO)	3
2.1 Class dan Object	3
2.2 Inheritance (Pewarisan).....	3
2.3 Polymorphism.....	4
2.4 Encapsulation (Enkapsulasi)	4
2.5 Abstract Class	5
2.6 Package	5
2.7 Collection	6
2.8 String Method.....	6
2.9 Exception Handling	7
2.10 File Handling	7
2.11 Database (JDBC)	8
BAB III REFLEKSI PENGEMBANGAN	9
3.1 Alasan Pemilihan Studi Kasus	9
3.2 Bagian Program yang Paling Sulit Dibuat	9
3.3 Kesalahan yang Paling Sering Ditemui dan Solusinya.....	9
3.4 Rencana Pengembangan Fitur Selanjutnya	10
3.5 Konsep PBO yang Masih Perlu Dipelajari Lebih Lanjut.....	10
LAMPIRAN.....	11
Lampiran A. Class Diagram.....	11
Lampiran B. Struktur Database	11
Lampiran C. Screenshot / Rancangan Tampilan Aplikasi	13
Lampiran D. Penjelasan Singkat Setiap Menu.....	16

DAFTAR TABEL

Tabel B.1 Rincian Struktur Tabel Database klinik_db	12
Tabel D.1 Penjelasan Singkat Setiap Menu.....	16

DAFTAR GAMBAR

Gambar A.1 Class Diagram Sistem Klinik Sederhana	11
Gambar B.1 Entity Relationship Diagram (ERD) klinik_db.....	11
Gambar C.1 Rancangan Tampilan Form Login	13
Gambar C.2 Rancangan Tampilan Menu Utama	13
Gambar C.3 Rancangan Tampilan Form Data Master Pasien	14
Gambar C.4 Rancangan Tampilan Form Transaksi Pemeriksaan.....	14
Gambar C.5 Rancangan Tampilan Form Laporan Pemeriksaan	15

BAB I

DESKRIPSI PROGRAM

1.1 Nama Aplikasi

Aplikasi yang dibangun diberi nama “Sistem Klinik Sederhana” (SIMKLINIK), yaitu aplikasi desktop berbasis Java Swing yang terhubung ke basis data MySQL untuk mengelola operasional administratif sebuah klinik kesehatan berskala kecil hingga menengah.

1.2 Tujuan Aplikasi

Aplikasi ini dibangun dengan tujuan:

1. Menyediakan sistem pencatatan data pasien, dokter, dan obat secara terkomputerisasi sehingga menggantikan pencatatan manual yang rawan kesalahan dan kehilangan data.
2. Mendukung proses transaksi pemeriksaan pasien, mulai dari pencatatan keluhan, diagnosa dokter, hingga peresepan obat, dalam satu alur kerja yang terintegrasi dengan basis data.
3. Menyediakan laporan sederhana atas transaksi pemeriksaan yang telah terjadi dalam suatu rentang tanggal, sebagai bahan evaluasi dan rekap operasional klinik.
4. Menjadi media penerapan seluruh konsep Pemrograman Berorientasi Objek (PBO) yang telah dipelajari selama satu semester, meliputi class dan object, inheritance, polymorphism, encapsulation, abstract class, package, collection, exception handling, file handling, dan koneksi database.

1.3 Permasalahan yang Diselesaikan

Pada klinik berskala kecil, pencatatan data pasien, riwayat pemeriksaan, dan pengelolaan stok obat sering kali masih dilakukan secara manual menggunakan buku catatan atau lembar kerja terpisah. Pendekatan ini menimbulkan beberapa permasalahan, di antaranya:

1. Data pasien dan riwayat pemeriksaan sulit dicari kembali secara cepat ketika pasien datang berobat ulang.
2. Tidak ada validasi otomatis terhadap stok obat, sehingga berpotensi terjadi peresepan obat yang stoknya sebenarnya sudah habis atau tidak mencukupi.
3. Perhitungan total biaya pemeriksaan (biaya periksa dokter ditambah harga obat yang diresepkan) rawan salah hitung apabila dilakukan secara manual.

4. Tidak tersedia rekap/laporan transaksi yang bisa difilter berdasarkan periode tanggal tertentu.
5. Tidak ada jejak audit (log) atas perubahan data yang dilakukan oleh operator/administrator klinik.

Aplikasi Sistem Klinik Sederhana menjawab permasalahan tersebut dengan menyediakan modul login, data master (Pasien, Dokter, Obat), modul transaksi Pemeriksaan yang otomatis mengurangi stok obat dan menghitung total biaya, modul Laporan dengan filter tanggal dan fitur ekspor ke berkas teks, serta pencatatan log aktivitas CRUD ke dalam berkas log_aktivitas.txt.

BAB II

ANALISIS KONSEP PEMROGRAMAN BERORIENTASI OBJEK (PBO)

Bagian ini menjelaskan bagaimana setiap konsep PBO diterapkan secara konkret pada source code aplikasi Sistem Klinik Sederhana, lengkap dengan lokasi penerapan, alasan penggunaan, dan manfaatnya terhadap aplikasi.

2.1 Class dan Object

Lokasi penerapan: Seluruh berkas pada paket com.klinik.model (Person, Pasien, Dokter, Obat, Pemeriksaan, DetailResep, User), com.klinik.dao, com.klinik.gui, dan com.klinik.util. Total terdapat lebih dari 20 class, jauh melampaui syarat minimal 5 class.

```
public class Pasien extends Person {  
    private String alamat;  
    ...  
}  
  
// pembuatan object baru  
Pasien p = new Pasien("Budi Santoso", "Jl. Mawar", "08123456789", tgl,  
    "Laki-laki");
```

Alasan penggunaan: Setiap entitas dunia nyata pada domain klinik (pasien, dokter, obat, pemeriksaan) direpresentasikan sebagai class tersendiri agar data dan perilaku (behavior) yang terkait dengannya terbungkus dalam satu kesatuan yang mudah dipahami dan dipelihara, sesuai prinsip dasar pemrograman berorientasi objek.

Manfaat terhadap aplikasi: Kode menjadi terstruktur, mudah ditelusuri, dan setiap objek (misalnya sebuah instance Pasien) dapat langsung dipakai berulang kali di berbagai bagian aplikasi (form GUI, DAO, laporan) tanpa duplikasi logika.

2.2 Inheritance (Pewarisan)

Lokasi penerapan: com/klinik/model/Person.java (superclass) diwariskan oleh com/klinik/model/Pasien.java dan com/klinik/model/Dokter.java.

```
public abstract class Person {  
    private int id;  
    private String nama;  
    public abstract String getInfo();  
}  
  
public class Pasien extends Person { ... }  
public class Dokter extends Person { ... }
```

Alasan penggunaan: Pasien dan Dokter memiliki atribut dasar yang identik, yaitu id dan nama, beserta perilaku dasar yang sama pula (menyimpan identitas). Alih-alih menuliskan field dan method yang sama di dua class terpisah, atribut dan method umum tersebut diangkat ke superclass Person, kemudian diwariskan ke Pasien dan Dokter melalui kata kunci extends.

Manfaat terhadap aplikasi: Mengurangi duplikasi kode (DRY – Don't Repeat Yourself), memudahkan pemeliharaan karena perubahan pada atribut dasar (id, nama) cukup dilakukan satu kali di Person, serta membuka jalan bagi penerapan polymorphism melalui method abstrak getInfo().

2.3 Polymorphism

Lokasi penerapan: Method getInfo() pada Person.java (dideklarasikan abstract) yang di-override secara berbeda pada Pasien.java dan Dokter.java. Selain itu, seluruh class DAO (PasienDAO, DokterDAO, ObatDAO) juga menunjukkan polymorphism melalui implementasi interface Cruddable<T> pada com/klinik/dao/.

```
// Pasien.java
@Override
public String getInfo() {
    return "Pasien #" + getId() + " - " + getNama() + " (" +
    jenisKelamin + ")";
}

// Dokter.java
@Override
public String getInfo() {
    return "dr. " + getNama() + " - Spesialis " + spesialisasi;
}
```

Alasan penggunaan: Setiap subclass dari Person perlu menampilkan informasi ringkas dirinya dengan format yang berbeda: Pasien menampilkan nomor pasien dan jenis kelamin, sedangkan Dokter menampilkan gelar 'dr.' beserta spesialisasinya. Polymorphism memungkinkan satu nama method (getInfo) dipanggil pada objek Person mana pun (baik itu Pasien maupun Dokter), namun implementasi yang dijalankan otomatis menyesuaikan tipe objek sesungguhnya (runtime polymorphism / method overriding).

Manfaat terhadap aplikasi: Kode pemanggil cukup memanggil person.getInfo() tanpa perlu mengetahui atau memeriksa apakah objeknya Pasien atau Dokter, sehingga aplikasi lebih fleksibel dan mudah dikembangkan jika suatu saat ditambahkan subclass Person baru (misalnya Perawat) tanpa mengubah kode yang sudah ada.

2.4 Encapsulation (Enkapsulasi)

Lokasi penerapan: Diterapkan konsisten pada seluruh class model (Person, Pasien, Dokter, Obat, Pemeriksaan, DetailResep, User) di paket com.klinik.model.

```
private double harga;

public void setHarga(double harga) {
    if (harga < 0) throw new IllegalArgumentException("Harga tidak boleh minus");
    this.harga = harga;
}
```

Alasan penggunaan: Seluruh atribut/field pada class model dideklarasikan dengan modifier private, sehingga tidak dapat diakses langsung dari luar class. Akses dan perubahan nilai atribut hanya dapat dilakukan melalui method getter dan setter

publik. Beberapa setter bahkan menyertakan validasi, misalnya `setHarga()` pada class `Obat` yang menolak nilai harga negatif, dan `setNama()` pada `Person` yang menolak nama kosong.

Manfaat terhadap aplikasi: Melindungi integritas data dari perubahan nilai yang tidak valid (misalnya harga obat minus atau nama kosong), serta menyembunyikan detail implementasi internal class sehingga class lain hanya berinteraksi melalui antarmuka publik (getter/setter) yang jelas dan aman.

2.5 Abstract Class

Lokasi penerapan: `com/klinik/model/Person.java` dideklarasikan sebagai abstract class dengan satu method abstrak, yaitu `getInfo()`.

```
public abstract class Person {  
    ...  
    public abstract String getInfo();  
}  
  
// Person p = new Person(1, "Budi"); -> TIDAK VALID, error kompilasi
```

Alasan penggunaan: `Person` mewakili konsep umum 'orang' pada sistem klinik yang secara konseptual tidak pernah berdiri sendiri sebagai objek nyata (tidak pernah ada objek 'Person' murni di klinik, yang ada hanyalah Pasien atau Dokter). Oleh karena itu `Person` dijadikan abstract class agar tidak dapat diinstansiasi secara langsung (`new Person()` tidak diperbolehkan oleh compiler), namun tetap dapat mewariskan struktur dan kontrak method kepada subclass-nya.

Manfaat terhadap aplikasi: Memaksa setiap subclass (`Pasien`, `Dokter`) untuk menyediakan implementasinya sendiri atas method `getInfo()`, sekaligus mencegah pembuatan objek `Person` yang tidak bermakna secara bisnis. Ini membuat desain kelas lebih merepresentasikan aturan domain klinik yang sesungguhnya.

2.6 Package

Lokasi penerapan: Kode program diorganisasikan ke dalam 6 package di bawah root `com.klinik`, yaitu: `com.klinik` (Main), `com.klinik.model`, `com.klinik.dao`, `com.klinik.gui`, `com.klinik.exception`, dan `com.klinik.util`.

<code>com.klinik</code>	-> <code>Main.java</code>
<code>com.klinik.model</code>	-> <code>Person</code> , <code>Pasien</code> , <code>Dokter</code> , <code>Obat</code> , <code>Pemeriksaan</code> , <code>DetailResep</code> , <code>User</code>
<code>com.klinik.dao</code>	-> <code>Cruddable</code> , <code>PasienDAO</code> , <code>DokterDAO</code> , <code>ObatDAO</code> , <code>PemeriksaanDAO</code> , <code>UserDAO</code>
<code>com.klinik.gui</code>	-> <code>LoginForm</code> , <code>MenuUtama</code> , <code>FormPasien</code> , <code>FormDokter</code> , <code>FormObat</code> , <code>FormTransaksi</code> , <code>FormLaporan</code>
<code>com.klinik.exception</code>	-> <code>DataTidakDitemukanException</code> , <code>StokTidakCukupException</code>
<code>com.klinik.util</code>	-> <code>KoneksiDatabase</code> , <code>ValidasiInput</code> , <code>LogAktivitas</code>

Alasan penggunaan: Package digunakan untuk mengelompokkan class berdasarkan tanggung jawab dan lapisan arsitekturnya: model menyimpan representasi data, dao menangani akses ke database, gui berisi seluruh tampilan antarmuka pengguna, exception berisi exception kustom, dan util berisi kelas pendukung lintas-lapisan (koneksi database, validasi input, logging).

Manfaat terhadap aplikasi: Struktur proyek menjadi rapi dan mudah dinavigasi, menghindari bentrok nama class antar-lapisan, serta memisahkan tanggung jawab (separation of concerns) sehingga perubahan pada satu lapisan, misalnya tampilan GUI, tidak memengaruhi lapisan lain seperti akses data.

2.7 Collection

Lokasi penerapan: com/klinik/model/Pemeriksaan.java menggunakan List<DetailResep> (implementasi ArrayList) untuk menyimpan daftar resep obat dalam satu transaksi pemeriksaan. Collection serupa juga dipakai pada seluruh method tampilkanSemua() dan cari() di lapisan DAO yang mengembalikan List<Pasien>, List<Dokter>, List<Obat>, dan pada FormTransaksi.java yang menyimpan resepSementara sebagai List<DetailResep> sebelum transaksi disimpan.

```
private List<DetailResep> daftarResep = new ArrayList<>();

public double getTotalBiaya() {
    double total = biayaPeriksa;
    for (DetailResep r : daftarResep) {
        total += r.getSubtotal();
    }
    return total;
}
```

Alasan penggunaan: Satu kali pemeriksaan dapat menghasilkan lebih dari satu resep obat dengan jumlah yang tidak diketahui sebelumnya (dinamis), sehingga dibutuhkan struktur data yang dapat bertumbuh secara fleksibel. ArrayList dipilih karena mendukung penambahan elemen secara dinamis (add) serta iterasi yang mudah untuk menghitung total biaya.

Manfaat terhadap aplikasi: Memudahkan pengelolaan data yang jumlahnya tidak tetap tanpa perlu mendefinisikan ukuran array secara statis di awal, serta menyederhanakan proses perhitungan total biaya pemeriksaan dengan melakukan iterasi (for-each) atas seluruh resep dalam daftar.

2.8 String Method

Lokasi penerapan: com/klinik/util/ValidasiInput.java menggunakan berbagai method bawaan class String seperti trim(), isEmpty(), matches() (regular expression), toLowerCase(), split(), charAt(), toUpperCase(), dan substring().

```
public static boolean isNoTelpValid(String noTelp) {
    if (isKosong(noTelp)) return false;
    return noTelp.trim().matches("^([0-9]{10,13})$");
}

public static String toTitleCase(String teks) {
    String[] kata = teks.trim().toLowerCase().split("\\s+");
    ...
}
```

Alasan penggunaan: Input dari pengguna melalui form GUI (nama pasien, nomor telepon, dan sebagainya) perlu divalidasi dan dirapikan formatnya sebelum disimpan ke database, misalnya memastikan nomor telepon hanya berisi digit angka

dengan panjang tertentu, dan mengubah nama menjadi format Title Case yang konsisten.

Manfaat terhadap aplikasi: Mencegah data yang tidak konsisten atau tidak valid tersimpan ke database (misalnya nomor telepon berisi huruf), sekaligus menjaga keseragaman format tampilan nama pasien/dokter di seluruh aplikasi.

2.9 Exception Handling

Lokasi penerapan: Dua custom exception dibuat pada paket `com.klinik.exception`: `DataTidakDitemukanException.java` (dilempar oleh `UserDAO.login()` saat username/password salah) dan `StokTidakCukupException.java` (dilempar oleh `ObatDAO.kurangiStok()` saat stok obat tidak mencukupi). Keduanya ditangani menggunakan blok try-catch di lapisan GUI, misalnya pada `LoginForm.java` dan `FormTransaksi.java`, di samping penanganan `SQLException` bawaan Java pada seluruh operasi database.

```
public void kurangiStok(int idObat, int jumlah) throws SQLException,
StokTidakCukupException {
    ...
    if (rs.next() && rs.getInt("stok") < jumlah) {
        throw new StokTidakCukupException("Stok obat tidak mencukupi
untuk id_obat=" + idObat);
    }
    ...
}

} catch (SQLException | StokTidakCukupException e) {
    if (conn != null) conn.rollback();
    throw e;
}
```

Alasan penggunaan: Kondisi seperti login gagal atau stok obat tidak mencukupi bukanlah error pemrograman (bug), melainkan kondisi bisnis yang wajar terjadi dan perlu ditangani secara eksplisit dengan pesan yang informatif bagi pengguna, alih-alih membiarkan aplikasi crash. Membuat custom exception juga membuat maksud error lebih jelas dibandingkan hanya melempar Exception generik.

Manfaat terhadap aplikasi: Aplikasi menjadi lebih tangguh (robust) karena kondisi kegagalan ditangani dengan baik dan ditampilkan sebagai pesan yang mudah dipahami pengguna melalui `JOptionPane`, bukan berupa stack trace teknis yang membingungkan. Penggunaan transaction rollback pada `PemeriksaanDAO` juga memastikan data tidak tersimpan sebagian (partial) apabila terjadi `StokTidakCukupException` di tengah proses.

2.10 File Handling

Lokasi penerapan: `com/klinik/util/LogAktivitas.java` menulis setiap aktivitas tambah/ubah/hapus data ke berkas `log_aktivitas.txt` menggunakan `FileWriter` dan `PrintWriter` (append mode). Selain itu, `com/klinik/gui/FormLaporan.java` menyediakan fitur ekspor laporan pemeriksaan ke berkas teks (`laporan_pemeriksaan_<tanggal>.txt`) melalui method `exportKeFile()`.

```
public static void catat(String aktivitas) {
    String waktu =
```

```

LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd-MM-yyyy
HH:mm:ss"));
    try (FileWriter fw = new FileWriter("log_aktivitas.txt", true);
        PrintWriter pw = new PrintWriter(fw)) {
        pw.println "[" + waktu + " ] " + aktivitas);
    } catch (IOException e) { ... }
}

```

Alasan penggunaan: Klinik memerlukan jejak audit (audit trail) sederhana atas siapa melakukan perubahan data apa dan kapan, meskipun aplikasi ini belum memiliki tabel log di database. Menyimpan log ke berkas teks merupakan solusi ringan dan mudah diperiksa sewaktu-waktu. Fitur ekspor laporan ke berkas .txt juga memudahkan staf klinik untuk mencetak atau mengarsipkan laporan pemeriksaan di luar aplikasi.

Manfaat terhadap aplikasi: Memberikan kemampuan audit dasar tanpa menambah kompleksitas skema database, serta memberi pengguna cara mudah membawa data laporan keluar dari aplikasi (portable, dapat dibuka di Notepad atau diprint) tanpa memerlukan aplikasi khusus.

2.11 Database (JDBC)

Lokasi penerapan: com/klinik/util/KoneksiDatabase.java mengelola koneksi ke MySQL menggunakan JDBC (pola singleton). Seluruh class pada paket com.klinik.dao (PasienDAO, DokterDAO, ObatDAO, PemeriksaanDAO, UserDAO) menggunakan PreparedStatement untuk operasi CRUD dan pencarian data ke 6 tabel pada database klinik_db (user, dokter, pasien, obat, pemeriksaan, detail_resep) — melampaui syarat minimal 4 tabel.

```

public static Connection getConnection() throws SQLException {
    if (koneksi == null || koneksi.isClosed()) {
        Class.forName("com.mysql.cj.jdbc.Driver");
        koneksi = DriverManager.getConnection(URL, USER, PASSWORD);
    }
    return koneksi;
}

String sql = "SELECT * FROM pasien WHERE nama LIKE ? OR no_telp LIKE ?";
PreparedStatement ps = conn.prepareStatement(sql);

```

Alasan penggunaan: Data klinik (pasien, dokter, obat, riwayat pemeriksaan) bersifat permanen dan harus tetap tersimpan meskipun aplikasi ditutup, serta berpotensi diakses oleh banyak pengguna/komputer secara bersamaan di lingkungan klinik nyata. MySQL sebagai basis data relasional dipilih karena mendukung relasi antar tabel (foreign key) yang sesuai dengan hubungan data pada domain klinik, misalnya satu pemeriksaan terhubung ke satu pasien dan satu dokter. PreparedStatement dipakai (bukan Statement biasa) untuk mencegah celah keamanan SQL Injection.

Manfaat terhadap aplikasi: Data tersimpan secara persisten, konsisten, dan dapat diquery secara efisien (misalnya pencarian pasien berdasarkan nama, atau laporan pemeriksaan berdasarkan rentang tanggal menggunakan klausa SQL BETWEEN dan JOIN antar tabel pemeriksaan, pasien, dan dokter).

BAB III

REFLEKSI PENGEMBANGAN

3.1 Alasan Pemilihan Studi Kasus

Studi kasus Sistem Klinik Sederhana dipilih karena domainnya cukup familiar dan dekat dengan kehidupan sehari-hari, sehingga memudahkan dalam merancang entitas dan alur bisnis yang realistis. Selain itu, domain klinik secara alami memiliki relasi antar data yang jelas (pasien, dokter, obat, dan transaksi pemeriksaan yang menghubungkan ketiganya), sehingga cocok digunakan untuk menunjukkan penerapan hampir seluruh konsep PBO secara bermakna, mulai dari inheritance (Pasien dan Dokter sama-sama seorang Person), hingga exception handling yang relevan secara bisnis (misalnya validasi stok obat).

3.2 Bagian Program yang Paling Sulit Dibuat

Bagian yang paling menantang adalah modul Transaksi Pemeriksaan (FormTransaksi.java dan PemeriksaanDAO.java), karena satu transaksi pemeriksaan melibatkan penyimpanan ke dua tabel sekaligus secara konsisten, yaitu tabel pemeriksaan (header) dan tabel detail_resep (detail obat yang diresepkan), sekaligus harus mengurangi stok obat pada tabel obat. Kesulitan utamanya adalah memastikan operasi tersebut bersifat atomik: apabila salah satu obat pada resep ternyata stoknya tidak mencukupi di tengah proses, seluruh transaksi (termasuk data header pemeriksaan yang sudah sempat disimpan) harus dibatalkan agar data tidak tersimpan setengah-setengah. Hal ini diselesaikan dengan menerapkan transaction database (conn.setAutoCommit(false) beserta conn.commit() dan conn.rollback()) pada method simpanTransaksi().

3.3 Kesalahan yang Paling Sering Ditemui dan Solusinya

Beberapa error yang paling sering muncul selama pengembangan antara lain:

1. SQLException akibat konfigurasi koneksi database yang belum sesuai (URL, username, password pada KoneksiDatabase.java, atau driver MySQL Connector/J yang belum ditambahkan ke classpath/library project) — diatasi dengan memeriksa kembali konfigurasi koneksi dan memastikan berkas mysql-connector-j-x.x.x.jar sudah ditambahkan pada folder lib/ serta classpath proyek.
2. NullPointerException ketika combo box (misalnya daftar dokter atau obat pada FormTransaksi) masih kosong karena data master belum diisi terlebih dahulu — diatasi dengan menambahkan pengecekan null sebelum data diproses dan menampilkan pesan peringatan kepada pengguna.
3. NumberFormatException ketika pengguna memasukkan teks non-angka pada kolom yang seharusnya numerik, seperti jumlah obat atau biaya periksa —

diatasi dengan membungkus proses parsing (`Integer.parseInt` / `Double.parseDouble`) dalam blok try-catch dan menampilkan pesan validasi yang jelas.

4. Kesalahan format tanggal (`DateTimeParseException`) pada input tanggal lahir atau filter laporan yang tidak mengikuti format yyyy-MM-dd yang diharapkan oleh `LocalDate.parse()` — diatasi dengan memberi contoh format pada label input dan nilai default yang sudah berformat benar.

3.4 Rencana Pengembangan Fitur Selanjutnya

Tiga fitur yang ingin ditambahkan jika aplikasi dikembangkan kembali:

1. Manajemen hak akses (role-based access control) yang lebih lengkap, sehingga peran seperti Admin, Dokter, dan Kasir memiliki menu dan kewenangan yang berbeda, tidak hanya menyimpan kolom role tanpa pembatasan akses nyata seperti pada versi saat ini.
2. Fitur cetak struk/nota pemeriksaan dan resep obat dalam format PDF, sehingga pasien dapat menerima bukti pemeriksaan secara fisik, memanfaatkan library seperti `iText` atau `JasperReports`.
3. Dashboard statistik ringkas pada Menu Utama (misalnya grafik jumlah kunjungan pasien per bulan atau obat dengan stok menipis), memanfaatkan library chart seperti `JFreeChart`, untuk membantu pengambilan keputusan operasional klinik.

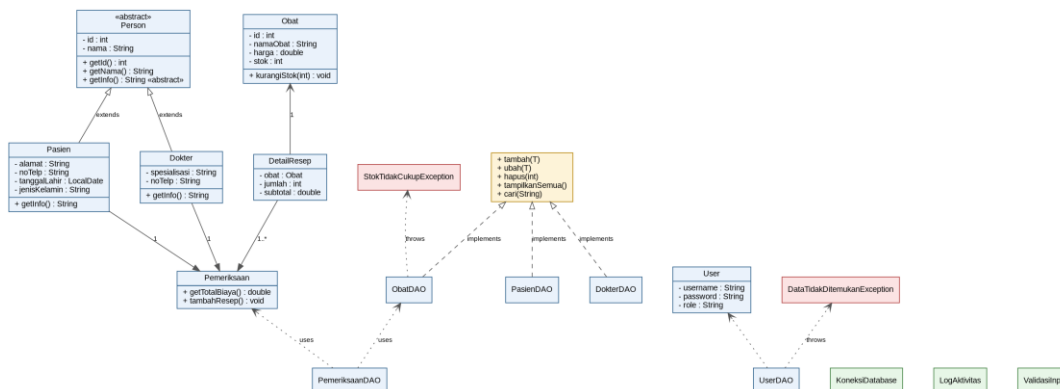
3.5 Konsep PBO yang Masih Perlu Dipelajari Lebih Lanjut

Konsep design pattern lanjutan (seperti `DAO Factory`, `Observer`, dan `MVC` secara lebih ketat) masih perlu didalami lebih jauh, karena pada aplikasi ini lapisan GUI (Form) masih memanggil DAO secara langsung tanpa lapisan `Service/Controller` terpisah. Selain itu, pemahaman mengenai connection pooling (misalnya menggunakan `HikariCP`) untuk koneksi database yang lebih efisien pada aplikasi multi-pengguna, serta penerapan generic method yang lebih kompleks pada interface `Crudable<T>`, juga menjadi topik yang ingin dipelajari lebih lanjut ke depannya.

LAMPIRAN

Lampiran A. Class Diagram

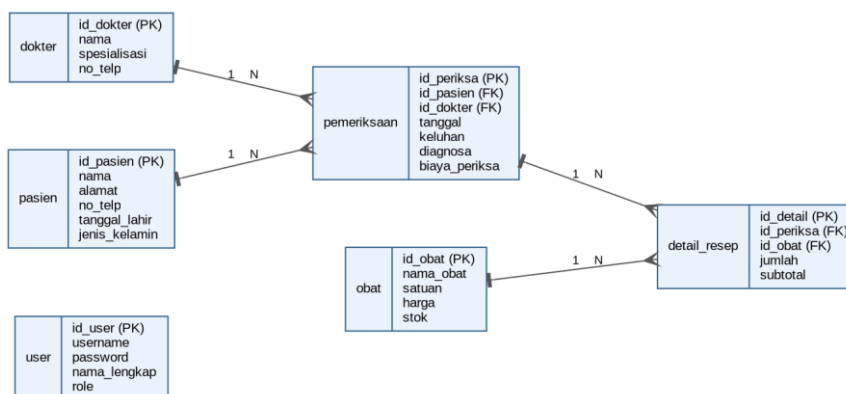
Class diagram berikut menggambarkan struktur class utama pada aplikasi Sistem Klinik Sederhana beserta relasi inheritance (extends), realisasi interface (implements), dan relasi asosiasi/komposisi antar class.



Gambar A.1 – Class Diagram Sistem Klinik Sederhana

Lampiran B. Struktur Database

Database klinik_db terdiri atas 6 tabel yang saling berelasi melalui foreign key, sebagaimana digambarkan pada Entity Relationship Diagram (ERD) berikut.



Gambar B.1 – Entity Relationship Diagram (ERD) klinik_db

Rincian struktur setiap tabel adalah sebagai berikut:

Tabel	Kolom Utama	Keterangan
user	id_user (PK), username, password, nama_lengkap, role	Menyimpan akun untuk login aplikasi
dokter	id_dokter (PK), nama, spesialisasi, no_telp	Data master dokter
pasien	id_pasien (PK), nama, alamat, no_telp, tanggal_lahir, jenis_kelamin	Data master pasien
obat	id_obat (PK), nama_obat, satuan, harga, stok	Data master obat & stok

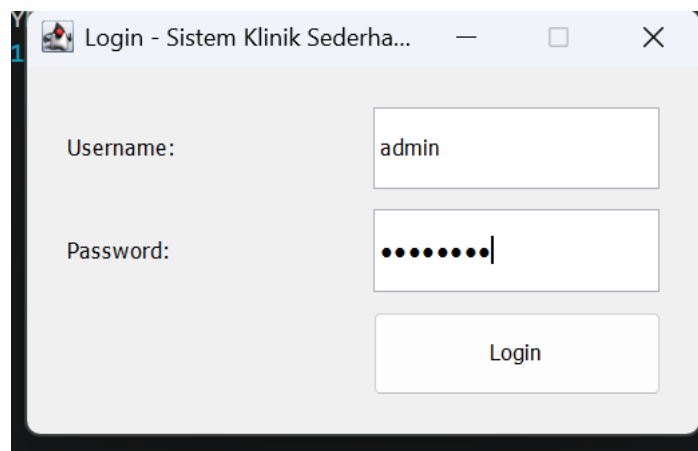
Tabel	Kolom Utama	Keterangan
pemeriksaan	id_periksa (PK), id_pasien (FK), id_dokter (FK), tanggal, keluhan, diagnosa, biaya_periksa	Header transaksi pemeriksaan
detail_resep	id_detail (PK), id_periksa (FK), id_obat (FK), jumlah, subtotal	Detail resep obat per transaksi

Tabel B.1 – Rincian Struktur Tabel Database klinik_db

Lampiran C. Screenshot / Rancangan Tampilan Aplikasi

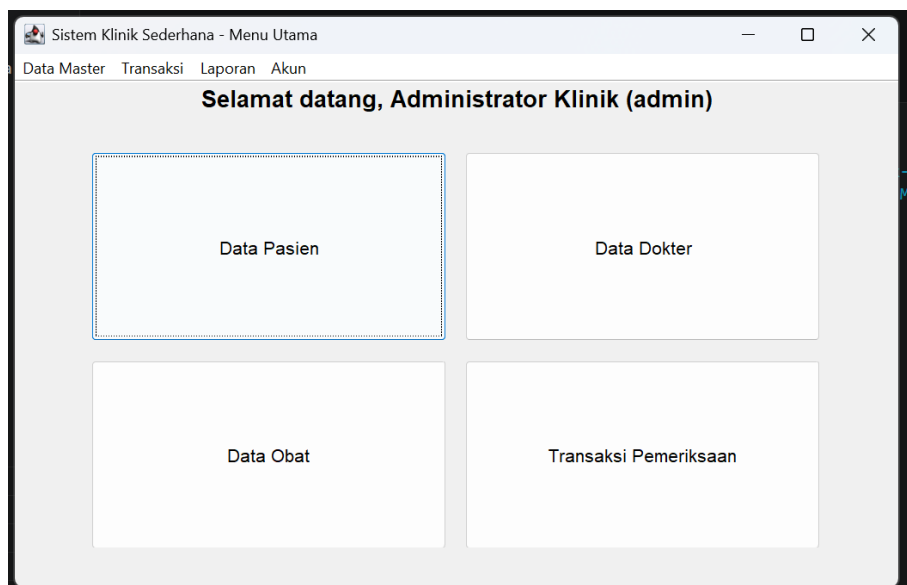
Catatan: gambar pada subbagian ini merupakan rancangan/wireframe tampilan yang disusun berdasarkan struktur GUI pada kode program (tata letak komponen Swing seperti JLabel, JTextField, JTable, dan JButton sesuai berkas *.java pada paket com.klinik.gui). Sebelum laporan ini dikumpulkan, disarankan untuk menggantinya dengan tangkapan layar (screenshot) asli hasil menjalankan aplikasi pada perangkat yang telah terhubung ke database MySQL.

a. Form Login



Gambar C.1 – tampilan Form Login

b. Menu Utama



Gambar C.2 – tampilan Menu Utama

c. Form Data Master (contoh: Data Pasien)

Data Master Pasien

Form Data Pasien

ID (auto): Nama:

Alamat: No. Telp:

Tgl Lahir (yyyy-mm-dd): Jenis Kelamin:

Cari:

ID	Nama	Alamat	No. Telp	Tgl Lahir	JK
1	Ari Rizal Zaaludin	CIAMIS	082537236325	2000-01-01	Laki-laki

Gambar C.3 – tampilan Form Data Master Pasien

d. Form Transaksi Pemeriksaan

Transaksi Pemeriksaan Pasien

Data Pemeriksaan

Pasien: Dokter:

Keluhan:

Diagnosa: Biaya Periksa:

Resep Obat

Obat: Jumlah:

Nama Obat	Jumlah	Harga Satuan	Subtotal
-----------	--------	--------------	----------

Gambar C.4 – tampilan Form Transaksi Pemeriksaan

e. Form Laporan

Laporan Pemeriksaan

Dari tanggal (yyyy-mm-dd): 2026-07-01 Sampai tanggal: 2026-07-25 Tampilkan

ID Periksa	Pasien	Dokter	Tanggal	Diagnosa	Biaya Periksa
------------	--------	--------	---------	----------	---------------

Export ke File (.txt)

Gambar C.5 – tampilan Form Laporan Pemeriksaan

Lampiran D. Penjelasan Singkat Setiap Menu

Menu / Form	Fungsi
Login	Autentikasi pengguna (username & password) sebelum masuk ke aplikasi; menampilkan pesan error melalui <code>DataTidakDitemukanException</code> jika kredensial salah.
Menu Utama	Halaman utama setelah login, berisi menu bar (Data Master, Transaksi, Laporan, Akun) serta tombol pintas ke masing-masing form.
Data Pasien / Data Dokter / Data Obat	Form Data Master untuk operasi CRUD (tambah, ubah, hapus, tampilkan) serta pencarian data pasien, dokter, dan obat.
Transaksi Pemeriksaan	Form Transaksi untuk mencatat pemeriksaan baru: memilih pasien & dokter, mengisi keluhan/diagnosa/biaya periksa, menambahkan resep obat, lalu menyimpan seluruh transaksi ke database sekaligus mengurangi stok obat.
Laporan Pemeriksaan	Form Laporan untuk menampilkan rekap transaksi pemeriksaan berdasarkan rentang tanggal, serta mengekspor hasilnya ke berkas teks (.txt).
Logout	Mengakhiri sesi pengguna dan kembali ke Form Login.

Tabel D.1 – Penjelasan Singkat Setiap Menu